

Interop Resolution Service

API reference for Onyx: member resolution, `POST /v1/interop/resolve`.

Version v1 · Point32Health · the OpenAPI document (YAML and JSON) that ships with this page carries the same contract.

1 Overview

Onyx's interop lookups. Member resolution, `POST /v1/interop/resolve`: resolves an EMR-supplied member ID against the plan's member records, reports whether coverage is active on the date of service, and returns the ID as resolved and in every UM vendor's format, each with the payer for that vendor's request.

One call per prior authorization. Send the member id exactly as the provider's EMR supplied it, the date of service (or the period of service) and, when known, the patient's date of birth. The answer says whether the member was found, whether coverage is active on that date, the line of business to route on, and the member id in every UM vendor's format with the payer to put on that vendor's request.

Base URL and credentials are issued by Point32Health for each environment and are not part of this document. The path below is relative to that base URL. The credential pack names the header the credential travels in; nothing in the body or the path carries a credential, and the service itself does not read it. A rejected credential is answered in front of the service (401 or 403) without the error envelope of section 6.

2 The operation

`POST /v1/interop/resolve` with `Content-Type: application/json`. The service always answers JSON, whatever `Accept` header is sent; every response carries `Cache-Control: no-store` and `X-Content-Type-Options: nosniff`.

Finds the member behind the id the EMR typed, says whether coverage is active on the date of service (or on every day of a period of service), and returns the id as resolved and in every configured UM vendor's format (memberId.forVendors), each entry with the payer for that vendor's request (payerId, payerName). When several members on the plan match and no dateOfBirth settles it, the outcome is AMBIGUOUS and the message says what to resend; the other members are never listed. clientId, clientType (EXT or INT), requestId and memberId are required; unknown properties are ignored. A sent date is the date evaluated; an unusable date is a 400. POST because the member id is PHI and must not appear in URLs; the operation is a pure read and may be repeated.

Headers

Header	Direction	Rule
X-Correlation-Id	in	Optional transport-level id, 1 to 64 characters of letters, digits, <code>.</code> , <code>_</code> , <code>:</code> or <code>-</code> ; opaque. Missing or invalid: the service generates one. The caller's own id for the call is <code>requestId</code> in the body.
X-Correlation-Id	out	The echoed or generated value; also in the body of error responses as <code>correlationId</code> .

Header	Direction	Rule
Retry-After	out	On 503 only: seconds to wait before retrying (10).

Responses

Status	Meaning
200	outcome ACTIVE INACTIVE NOT_FOUND AMBIGUOUS; memberId.forVendors, each vendor with its payer, present for ACTIVE and INACTIVE
400	INVALID_REQUEST for a missing or unusable clientId, clientType or requestId, a missing memberId, an unusable date or a body that is not JSON; MEMBER_LOOKUP_REJECTED when the member lookup answered 400 (details[0].code HTTP_400, the lookup's text in the message)
422	DOB_MISMATCH: the date of birth matches no record for this id
404	INVALID_REQUEST / ROUTE_NOT_FOUND: wrong path; a connector defect (a member that is not found is a 200 with outcome NOT_FOUND)
405	INVALID_REQUEST / METHOD_NOT_ALLOWED: the operation is POST only
415	INVALID_REQUEST / UNSUPPORTED_MEDIA_TYPE: the request body must be application/json
502	MEMBER_LOOKUP_ERROR or MEMBER_LOOKUP_INVALID_RESPONSE (the member lookup); MEMBER_PLAN_ERROR or MEMBER_PLAN_INVALID_RESPONSE (the member plan lookup)
503	MEMBER_LOOKUP_UNAVAILABLE or MEMBER_PLAN_UNAVAILABLE; retry after the Retry-After seconds; header Retry-After (Seconds to wait before retrying)
500	INTERNAL_ERROR: the service itself failed; retry once later, then alert the service owners with the correlationId and requestId

3 Request

Member resolution request: clientId, clientType, requestId and memberId are required; unknown properties are ignored; a sent date must be usable. Unknown properties are ignored, so a vendor sent by habit does no harm: the answer is always for every vendor.

Field	Type	Required	Meaning
clientId	string	yes	The calling system, as registered with the plan: letters, digits, '.', '_' or '-', at most 50 characters
clientType	EXT INT	yes	EXT for a caller outside the plan, INT for an internal one
requestId	string	yes	The caller's id for this call, unique per request; echoed in the response and in every error response. Letters, digits, '.', '_', ':' or '-', at most 64 characters
memberId	string	yes	The member id exactly as the EMR supplied it. Looked up unchanged (surrounding whitespace removed); the service never checks its shape, length or characters. For instance a THP Medicare card number is a letter and 8 digits; the example is that pattern, masked

Field	Type	Required	Meaning
<code>dateOfService</code>	string		Date of service, yyyy-MM-dd; the first date when the service covers a period. Omitted, null or blank: today in US Eastern time (America/New_York) is used, <code>dateOfServiceDefaulted</code> is true and the answer's <code>dateOfService</code> says which date was used; send the date whenever you have it. A sent value is the date evaluated: any future date is judged on the coverage on record; not a real date (DATE_OF_SERVICE_INVALID), or more than 10 years back (DATE_OF_SERVICE_OUT_OF_RANGE), is a 400.
<code>dateOfServiceEnd</code>	string		Optional last date of service, yyyy-MM-dd, when the service covers a period: coverage must hold on every day from <code>dateOfService</code> to it. Not before <code>dateOfService</code> ; needs <code>dateOfService</code> . Omitted, null or blank counts as not sent.
<code>dateOfBirth</code>	string		Optional patient date of birth, yyyy-MM-dd, used only to verify the member or to tell apart the members on a plan who share the id. Omitted, null or blank counts as not sent. Not a real date is a 400 (DATE_OF_BIRTH_INVALID); in the future or more than 125 years ago is a 400 (DATE_OF_BIRTH_OUT_OF_RANGE). Never logged, never echoed.

4 Response (200)

The answer: the outcome Onyx branches on, one sentence for a person, the member id as received, as resolved and in every vendor's format with that vendor's payer, the line of business, the date evaluated, the coverage flag with its period, the caller's requestId and a traceId for support. A field that does not apply is absent, never null. Fields are sent in the order listed.

Field	Type	Required	Meaning
<code>outcome</code>	ACTIVE INACTIVE NOT_FOUND AMBIGUOUS	yes	ACTIVE: member found, coverage active on the date of service (or on every day of the period). INACTIVE: member found, not covered on that date or not on every day of the period; message says why. NOT_FOUND: no member for this id on that date. AMBIGUOUS: several members match and nothing sent tells them apart; message says what to resend
<code>message</code>	string	yes	One sentence for a person reading the answer. Branch on outcome, not on this text
<code>memberId</code>	object	yes	The member id as received, as resolved, and in every vendor's format
<code>memberId.received</code>	string	yes	The member id as sent, surrounding whitespace removed
<code>memberId.resolved</code>	string		The id the lookup resolved to, as the plan stores it; for a converted member a different number from the one received. ACTIVE and INACTIVE only
<code>memberId.forVendors[]</code>	array		The resolved id in every configured vendor's format, each with the payer for that vendor's request; one entry per vendor, sorted by vendor code. ACTIVE and INACTIVE only

Field	Type	Required	Meaning
<code>memberId.forVendors[].vendor</code>	CARELON EVICORE EVOLENT MHK ONYX OPTUM		The vendor code
<code>memberId.forVendors[].memberId</code>	string		The resolved id in this vendor's format: the member id to put on this vendor's request
<code>memberId.forVendors[].payerId</code>	string		The payer id to put on this vendor's request; it differs by vendor and by the member's heritage company, so use the entry's value as given
<code>memberId.forVendors[].payerName</code>	string		The payer name that goes with payerId
<code>lineOfBusiness</code>	string		The line of business Onyx routes on, from the plan in effect on the date of service: SCO, MA-HMO, MA-PPO, D-SNP, MA-TOGETHER, RI-TOGETHER or MA-QHP. When no rule applies, the member record's own category is passed through (MCR, PP, COM today; other values may appear), and the field is absent when the record carries none. ACTIVE and INACTIVE only. Agree with Point32Health how to route a value outside the list, or an absent one
<code>dateOfService</code>	date (yyyy-MM-dd)	yes	The date evaluated; the first date when a period was asked about
<code>dateOfServiceEnd</code>	date (yyyy-MM-dd)		The last date evaluated; present only when the request asked about a period
<code>dateOfServiceDefaulted</code>	boolean	yes	true when no dateOfService was sent and today was used
<code>coverage</code>	object		The coverage flag and the coverage period behind it. ACTIVE and INACTIVE only
<code>coverage.coverageId</code>	string		An id for the coverage period: the resolved id reduced to letters and digits, then the effective and end dates as yyyyMMdd, run together with no separator (39991231 for an open-ended period). Present whenever the dates are; the same member with the same coverage always gets the same value
<code>coverage.active</code>	boolean	yes	The flag: true when coverage holds on the date of service, or on every day of the period of service
<code>coverage.effectiveDate</code>	date (yyyy-MM-dd)		First day of the coverage period
<code>coverage.endDate</code>	date (yyyy-MM-dd)		Last day of the coverage period; 3999-12-31 for an open-ended period
<code>requestId</code>	string	yes	The caller's requestId, echoed
<code>traceId</code>	string	yes	The id of the member lookup behind this answer; quote it to support
<code>sourceMessage</code>	object		What the member lookup said when it had no member for the id. NOT_FOUND only, and only when the lookup sent a message
<code>sourceMessage.type</code>	string		The message type as the lookup sent it
<code>sourceMessage.status</code>	string		The status as the lookup sent it

Field	Type	Required	Meaning
<code>sourceMessage.code</code>	string		The message code as the lookup sent it
<code>sourceMessage.text</code>	string		The message text as the lookup sent it

An AMBIGUOUS answer never lists the members who share the id.

5 What to do with each answer

Onyx branches on `outcome` for a 200, and on the HTTP status otherwise. Two rules for every row: Onyx never reformats a member id itself when the service is up, and Onyx keeps the full response with its transaction.

Signal	Meaning	Onyx action
200 <code>ACTIVE</code>	Member found; coverage active on the date of service, or on every day of the period of service (<code>coverage.active</code> is true).	Send each vendor its <code>memberId.forVendors</code> entry: the <code>memberId</code> of the entry whose <code>vendor</code> is that vendor, already in the vendor's format, with that entry's <code>payerId</code> and <code>payerName</code> on the request. Route on <code>lineOfBusiness</code> . Keep <code>coverage.coverageId</code> with the transaction as the reference to the coverage period behind the decision.
200 <code>INACTIVE</code>	Member found; not covered on that date, or not on every day of the period. <code>message</code> says why.	Hold for intake. <code>memberId.forVendors</code> and <code>lineOfBusiness</code> are still there if the business rule says to submit anyway.
200 <code>NOT_FOUND</code>	No member for this id on that date. <code>sourceMessage</code> carries what the member lookup said, when it sent a message.	"Member not found" worklist. If Onyx holds a fuller form of the id (for instance with the suffix), one resend with it.
200 <code>AMBIGUOUS</code>	Several members on the plan share the id and nothing sent tells them apart. Nobody is listed.	One resend with <code>dateOfBirth</code> or the member's full id including the suffix. If it is still AMBIGUOUS (twins share a date of birth, or the records carry none), intake takes the full id from the member's card and resends.
400 / 422	Request or identity problem (the error catalog below).	Never retry. Route by <code>error.code</code> and <code>details[].code</code> .
502 / 503 / 500	A lookup behind the service, or the service itself, could not answer.	Retry as section 8 says, always with a new <code>requestId</code> ; when the retries are used up, park the transaction and alert the service owners.
401 / 403 / 429	Answered by the API gateway in front of the service, without the error envelope.	401 and 403: a credential problem; never retry. 429: treat like a 503 and retry after its <code>Retry-After</code> .
No response at all (connection refused, timeout)	The service is down or unreachable.	Retry once with a new <code>requestId</code> , as section 8 says; still no response: pass the member id to the UM vendor exactly as the EMR supplied it, and record the <code>requestId</code> so the transaction can be reconciled later.

6 Errors

Every non-200 answer produced by the service has one shape. Answers produced by the API gateway in front of it (401, 403, 429, or any response whose body is not JSON) do not carry this envelope: treat 401 and 403 as a credential problem and never retry them; treat 429 like a 503 and retry after its `Retry-After`. `error.code` is stable and is what to branch on; `details[].code` refines it; `requestId` echoes the caller's id whenever it was usable, so a 400 about another field still carries it; `traceId` is present once the member lookup was made.

Field	Type	Required	Meaning
<code>error</code>	object	yes	The error: a stable code to branch on, a message for a person, and one detail per problem
<code>error.code</code>	string	yes	INVALID_REQUEST, MEMBER_LOOKUP_REJECTED, DOB_MISMATCH, MEMBER_LOOKUP_ERROR, MEMBER_LOOKUP_INVALID_RESPONSE, MEMBER_LOOKUP_UNAVAILABLE, MEMBER_PLAN_ERROR, MEMBER_PLAN_INVALID_RESPONSE, MEMBER_PLAN_UNAVAILABLE or INTERNAL_ERROR
<code>error.message</code>	string	yes	One sentence for a person
<code>error.details[]</code>	array		One entry per problem found; every problem with a request is reported together
<code>error.details[].field</code>	string		The request field the problem is about, when there is one
<code>error.details[].code</code>	string	yes	The detail code that refines error.code, for example MEMBER_ID_MISSING, DATE_OF_SERVICE_INVALID, HTTP_400, READ_TIMEOUT
<code>error.details[].message</code>	string	yes	One sentence for a person
<code>correlationId</code>	string	yes	The X-Correlation-Id of the call, echoed or generated
<code>requestId</code>	string		The caller's requestId, when the request carried a usable one
<code>traceId</code>	string		The id of the member lookup behind the answer, present once the lookup was made

Error catalog

HTTP	error.code	details[].code	Meaning and Onyx action
400	INVALID_REQUEST	CLIENT_ID_MISSING CLIENT_ID_INVALID CLIENT_TYPE_MISSING CLIENT_TYPE_INVALID REQUEST_ID_MISSING REQUEST_ID_INVALID MEMBER_ID_MISSING DATE_OF_SERVICE_INVALID DATE_OF_SERVICE_OUT_OF_RANGE DATE_OF_SERVICE_END_INVALID DATE_OF_SERVICE_END_BEFORE_START DATE_OF_SERVICE_END_WITHOUT_START DATE_OF_BIRTH_INVALID DATE_OF_BIRTH_OUT_OF_RANGE MALFORMED_JSON WRONG_JSON_TYPE	Never retry. Every problem with the request is reported together, one detail per problem. MEMBER_ID_MISSING and DATE_OF_SERVICE_OUT_OF_RANGE are provider data; the caller codes are configuration; the rest are mapping defects.
400	MEMBER_LOOKUP_REJECTED	HTTP_400	The member lookup could not process the request as sent; the message carries its text. Never retry; send the request and the message to the service owners.
422	DOB_MISMATCH	DOB_MISMATCH	The date of birth matches no record for this id. Manual identity review; never file the authorization.
404, 405, 415	INVALID_REQUEST	ROUTE_NOT_FOUND METHOD_NOT_ALLOWED UNSUPPORTED_MEDIA_TYPE	Wrong path, method or request content type: a connector defect. ROUTE_NOT_FOUND is the only 404 this service returns; a member that is not found is a 200 with outcome NOT_FOUND.
502	MEMBER_LOOKUP_ERROR MEMBER_LOOKUP_INVALID_RESPONSE	HTTP_<code> ERROR_MESSAGE UNPARSEABLE_BODY EMPTY_BODY NO_MEMBER_ID UNREADABLE_COVERAGE CLIENT_ERROR	The member lookup answered outside its contract, or with data the service could not read. Retry once with a new requestId; if it fails again, park the transaction and alert the service owners.
502	MEMBER_PLAN_ERROR MEMBER_PLAN_INVALID_RESPONSE	HTTP_<code> UNPARSEABLE_BODY EMPTY_BODY CLIENT_ERROR	The member plan lookup answered outside its contract, or with data the service could not read. Retry once with a new requestId; if it fails again, park the transaction and alert the service owners.
503	MEMBER_LOOKUP_UNAVAILABLE MEMBER_PLAN_UNAVAILABLE	CONNECT_FAILED READ_TIMEOUT HTTP_500 HTTP_5xx HTTP_429 HTTP_408	A lookup could not be reached, timed out or is temporarily unavailable. Retry after the Retry-After seconds (10) with a new requestId, as section 8 says; when the retries are used up, park the transaction and alert the service owners.

HTTP	error.code	details[].code	Meaning and Onyx action
500	INTERNAL_ERROR		The service itself failed. Retry once with a new requestId; if it fails again, park the transaction and alert the service owners with the correlationId and the requestId.

7 Examples

Real answers of the service against test data; the member ids and dates are fictitious. Where a `message` or `sourceMessage` forwards the lookup's own code and text, the example shows a placeholder, because the live values differ: branch on `error.code` and `details[].code` only.

Request

```
POST /v1/interop/resolve
Content-Type: application/json
X-Correlation-Id: ONYX-PA-2026-000123

{
  "clientId": "ONYX",
  "clientType": "EXT",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "memberId": "123456789",
  "dateOfService": "2026-10-15",
  "dateOfBirth": "1950-03-15"
}
```

200 ACTIVE, THP Medicare card number with the date of birth verified: the id in every vendor's format

```
{ "outcome": "ACTIVE",
  "message": "Member found; coverage active on 2026-10-15",
  "memberId": { "received": "123456789", "resolved": "123456789 01",
    "forVendors": [
      { "vendor": "CARELON", "memberId": "12345678901", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "EVICORE", "memberId": "12345678901", "payerId": "TUFTS",
        "payerName": "TUFTS" },
      { "vendor": "EVOLENT", "memberId": "12345678901", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "MHK", "memberId": "123456789 01", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "ONYX", "memberId": "12345678901", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "OPTUM", "memberId": "123456789", "payerId": "Point32Health",
        "payerName": "Point32Health" }
    ] },
  "lineOfBusiness": "MCR",
  "dateOfService": "2026-10-15", "dateOfServiceDefaulted": false,
  "coverage": { "coverageId": "123456789012021010139991231", "active": true,
    "effectiveDate": "2021-01-01", "endDate": "3999-12-31" },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852491-77574" }
```

The requests below carry the same `clientId`, `clientIdType` and `requestId`; only the member fields are shown.

200 ACTIVE, HPHC card form, no date of service sent: the resolved id on every entry

```
{ "memberId": "HP-456789012" }

{ "outcome": "ACTIVE",
  "message": "Member found; coverage active on 2026-10-05",
  "memberId": { "received": "HP-456789012", "resolved": "HP456789012",
    "forVendors": [
      { "vendor": "CARELON", "memberId": "HP456789012", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "EVICORE", "memberId": "HP456789012",
        "payerId": "HARVARD PILGRIM HEALTH", "payerName": "HARVARD PILGRIM HEALTH" },
      { "vendor": "EVOLENT", "memberId": "HP456789012", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "MHK", "memberId": "HP456789012", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "ONYX", "memberId": "HP456789012", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "OPTUM", "memberId": "HP456789012", "payerId": "Point32Health",
        "payerName": "Point32Health" }
    ] },
  "lineOfBusiness": "COM",
  "dateOfService": "2026-10-05", "dateOfServiceDefaulted": true,
  "coverage": { "coverageId": "HP4567890122013030139991231", "active": true,
    "effectiveDate": "2013-03-01", "endDate": "3999-12-31" },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852498-65261" }
```

200 INACTIVE, SCO member: the ids are still returned

```
{ "memberId": "234567890", "dateOfService": "2026-10-15" }

{ "outcome": "INACTIVE",
  "message": "Member found; coverage ended before 2026-10-15",
  "memberId": { "received": "234567890", "resolved": "234567890 01",
    "forVendors": [
      { "vendor": "CARELON", "memberId": "23456789001", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "EVICORE", "memberId": "23456789001", "payerId": "TUFTS",
        "payerName": "TUFTS" },
      { "vendor": "EVOLENT", "memberId": "23456789001", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "MHK", "memberId": "234567890 01", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "ONYX", "memberId": "23456789001", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "OPTUM", "memberId": "234567890", "payerId": "Point32Health",
        "payerName": "Point32Health" }
    ] },
  "lineOfBusiness": "MCR",
  "dateOfService": "2026-10-15", "dateOfServiceDefaulted": false,
  "coverage": { "active": false },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852504-23697" }
```

200 INACTIVE, Public Plans dependent in a gap

```
{ "memberId": "34567890102", "dateOfService": "2024-08-15" }

{ "outcome": "INACTIVE",
  "message": "Member found; no coverage on 2024-08-15 (gap between coverage periods)",
  "memberId": { "received": "34567890102", "resolved": "34567890102",
    "forVendors": [
      { "vendor": "CARELON", "memberId": "34567890102", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "EVICORE", "memberId": "34567890102", "payerId": "TUFTS",
        "payerName": "TUFTS" },
      { "vendor": "EVOLENT", "memberId": "34567890102", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "MHK", "memberId": "34567890102", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "ONYX", "memberId": "34567890102", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "OPTUM", "memberId": "34567890102", "payerId": "Point32Health",
        "payerName": "Point32Health" }
    ] },
  "lineOfBusiness": "PP",
  "dateOfService": "2024-08-15", "dateOfServiceDefaulted": false,
  "coverage": { "active": false },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852509-10102" }
```

200 AMBIGUOUS, 9-character Together family, no date of birth

```
{ "memberId": "345678901", "dateOfService": "2026-10-15" }

{ "outcome": "AMBIGUOUS",
  "message": "Several members match this id; resend with dateOfBirth or the member's full id including the suffix",
  "memberId": { "received": "345678901" },
  "dateOfService": "2026-10-15", "dateOfServiceDefaulted": false,
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791255194024-32990" }
```

200 NOT_FOUND, no date of service sent: the member lookup answered 404, forwarded in sourceMessage

```
{ "memberId": "HP111222333" }

{ "outcome": "NOT_FOUND",
  "message": "No member found for this id",
  "memberId": { "received": "HP111222333" },
  "dateOfService": "2026-10-05", "dateOfServiceDefaulted": true,
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852517-51447",
  "sourceMessage": { "type": "<as the lookup sent it>",
    "status": "<as the lookup sent it>", "code": "<as the lookup sent it>",
    "text": "<as the lookup sent it>" } }
```

200 NOT_FOUND, 10-digit id: sent to the member lookup as typed, the member lookup has no record

```
{ "memberId": "1234567890", "dateOfService": "2026-10-15" }

{ "outcome": "NOT_FOUND",
  "message": "No member found for this id",
  "memberId": { "received": "1234567890" },
  "dateOfService": "2026-10-15", "dateOfServiceDefaulted": false,
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852521-20476",
  "sourceMessage": { "type": "<as the lookup sent it>",
    "status": "<as the lookup sent it>", "code": "<as the lookup sent it>",
    "text": "<as the lookup sent it>" } }
```

200 ACTIVE, converted member, old THP id, date after migration

```
{ "memberId": "567890123 01", "dateOfService": "2026-10-15" }

{ "outcome": "ACTIVE",
  "message": "Member found; coverage active on 2026-10-15",
  "memberId": { "received": "567890123 01", "resolved": "HP567890123",
    "forVendors": [
      { "vendor": "CARELON", "memberId": "HP567890123", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "EVICORE", "memberId": "HP567890123",
        "payerId": "HARVARD PILGRIM HEALTH", "payerName": "HARVARD PILGRIM HEALTH" },
      { "vendor": "EVOLENT", "memberId": "HP567890123", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "MHK", "memberId": "HP567890123", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "ONYX", "memberId": "HP567890123", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "OPTUM", "memberId": "HP567890123", "payerId": "Point32Health",
        "payerName": "Point32Health" }
    ] },
  "lineOfBusiness": "COM",
  "dateOfService": "2026-10-15", "dateOfServiceDefaulted": false,
  "coverage": { "coverageId": "HP5678901232020010139991231", "active": true,
    "effectiveDate": "2020-01-01", "endDate": "3999-12-31" },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852525-35782" }
```

422 DOB_MISMATCH

```
{ "error": { "code": "DOB_MISMATCH",
  "message": "dateOfBirth does not match the plan record(s) for this member id",
  "details": [
    { "field": "dateOfBirth", "code": "DOB_MISMATCH",
      "message": "no returned record has this date of birth" } ] },
  "correlationId": "ONYX-PA-2026-000123",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852529-13534" }
```

503 MEMBER_LOOKUP_UNAVAILABLE: the member lookup answered 500, its code and text forwarded, Retry-After: 10

```
{ "error": { "code": "MEMBER_LOOKUP_UNAVAILABLE",
  "message": "The member lookup reported an internal error: <the lookup's code and text>",
  "details": [
    { "code": "HTTP_500",
      "message": "The member lookup reported an internal error: <the lookup's code and text>" } ] },
  "correlationId": "ONYX-PA-2026-000123",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852533-40365" }
```

400 MEMBER_LOOKUP_REJECTED: the member lookup itself answered 400; its code and text forwarded, no Retry-After

```
{ "error": { "code": "MEMBER_LOOKUP_REJECTED",
  "message": "The member lookup rejected the request: <the lookup's code and text>",
  "details": [
    { "code": "HTTP_400",
      "message": "The member lookup rejected the request: <the lookup's code and text>" } ] },
  "correlationId": "ONYX-PA-2026-000123",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852536-12395" }
```

502 MEMBER_LOOKUP_ERROR / ERROR_MESSAGE: the member lookup reported an error and no member; its code and text forwarded

```
{ "error": { "code": "MEMBER_LOOKUP_ERROR",
  "message": "The member lookup reported an error: <the lookup's code and text>",
  "details": [
    { "code": "ERROR_MESSAGE",
      "message": "The member lookup reported an error: <the lookup's code and text>" } ] },
  "correlationId": "ONYX-PA-2026-000123",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852539-58663" }
```

400 INVALID_REQUEST, two problems at once

```
{ "memberId": "", "dateOfService": "10/15/2026" }

{ "error": { "code": "INVALID_REQUEST", "message": "request rejected",
  "details": [
    { "field": "memberId", "code": "MEMBER_ID_MISSING",
      "message": "memberId is required" },
    { "field": "dateOfService", "code": "DATE_OF_SERVICE_INVALID",
      "message": "expected a real date in yyyy-MM-dd" } ] },
  "correlationId": "ONYX-PA-2026-000123",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13" }
```

400 INVALID_REQUEST, the caller not identified: no clientId, clientType or requestId, so nothing to echo

```
{ "error": { "code": "INVALID_REQUEST", "message": "request rejected",
  "details": [
    { "field": "clientId", "code": "CLIENT_ID_MISSING",
      "message": "clientId is required" },
    { "field": "clientType", "code": "CLIENT_TYPE_MISSING",
      "message": "clientType is required: EXT or INT" },
    { "field": "requestId", "code": "REQUEST_ID_MISSING",
      "message": "requestId is required: the caller's id for this call" } ] },
  "correlationId": "ONYX-PA-2026-000123" }
```

200 INACTIVE for a 2027 date of service on a record that ends 12/31/2026; a vendor and an unknown property in the request, both ignored

```
{ "memberId": "T20262026", "dateOfService": "2027-10-15", "vendor": "EVICORE",  
  "anything": "goes" }  
  
{ "outcome": "INACTIVE",  
  "message": "Member found; coverage ended before 2027-10-15",  
  "memberId": { "received": "T20262026", "resolved": "T20262026  01",  
    "forVendors": [  
      { "vendor": "CARELON", "memberId": "T2026202601", "payerId": "Point32Health",  
        "payerName": "Point32Health" },  
      { "vendor": "EVICORE", "memberId": "T2026202601", "payerId": "TUFTS",  
        "payerName": "TUFTS" },  
      { "vendor": "EVOLENT", "memberId": "T2026202601", "payerId": "Point32Health",  
        "payerName": "Point32Health" },  
      { "vendor": "MHK", "memberId": "T20262026  01", "payerId": "Point32Health",  
        "payerName": "Point32Health" },  
      { "vendor": "ONYX", "memberId": "T2026202601", "payerId": "Point32Health",  
        "payerName": "Point32Health" },  
      { "vendor": "OPTUM", "memberId": "T20262026", "payerId": "Point32Health",  
        "payerName": "Point32Health" }  
    ] },  
  "lineOfBusiness": "MCR",  
  "dateOfService": "2027-10-15", "dateOfServiceDefaulted": false,  
  "coverage": { "active": false },  
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",  
  "traceId": "INTEROP-1791253852548-01454" }
```

The date judged is the date that was sent. This test member's only coverage period is 2026-01-01 to 2026-12-31, so 2026-10-15 is ACTIVE and 2027-10-15 is INACTIVE. An open-ended period (endDate 3999-12-31, see coverage.endDate in section 4) would make a 2027 date ACTIVE, because that is what the record says.

200 ACTIVE for a period of service across two plan-year records

```
{ "memberId": "U20252026", "dateOfService": "2025-12-20", "dateOfServiceEnd": "2026-01-05" }

{ "outcome": "ACTIVE",
  "message": "Member found; coverage active from 2025-12-20 to 2026-01-05",
  "memberId": { "received": "U20252026", "resolved": "U20252026 01",
    "forVendors": [
      { "vendor": "CARELON", "memberId": "U2025202601", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "EVICORE", "memberId": "U2025202601", "payerId": "TUFTS",
        "payerName": "TUFTS" },
      { "vendor": "EVOLENT", "memberId": "U2025202601", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "MHK", "memberId": "U20252026 01", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "ONYX", "memberId": "U2025202601", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "OPTUM", "memberId": "U20252026", "payerId": "Point32Health",
        "payerName": "Point32Health" }
    ] },
  "lineOfBusiness": "MCR",
  "dateOfService": "2025-12-20", "dateOfServiceEnd": "2026-01-05",
  "dateOfServiceDefaulted": false,
  "coverage": { "coverageId": "U20252026012025010120261231", "active": true,
    "effectiveDate": "2025-01-01", "endDate": "2026-12-31" },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852552-13365" }
```

This test member has one record for 2025 and one for 2026. They touch, so they are one continuous period, 2025-01-01 to 2026-12-31, and the period of service inside it is ACTIVE.

200 INACTIVE for a period of service that outlasts the coverage

```
{ "memberId": "U20252026", "dateOfService": "2026-12-20", "dateOfServiceEnd": "2027-01-05" }

{ "outcome": "INACTIVE",
  "message": "Member found; coverage active on 2026-12-20 but ends 2026-12-31, before 2027-01-05",
  "memberId": { "received": "U20252026", "resolved": "U20252026 01",
    "forVendors": [
      { "vendor": "CARELON", "memberId": "U2025202601", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "EVICORE", "memberId": "U2025202601", "payerId": "TUFTS",
        "payerName": "TUFTS" },
      { "vendor": "EVOLENT", "memberId": "U2025202601", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "MHK", "memberId": "U20252026 01", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "ONYX", "memberId": "U2025202601", "payerId": "Point32Health",
        "payerName": "Point32Health" },
      { "vendor": "OPTUM", "memberId": "U20252026", "payerId": "Point32Health",
        "payerName": "Point32Health" }
    ] },
  "lineOfBusiness": "MCR",
  "dateOfService": "2026-12-20", "dateOfServiceEnd": "2027-01-05",
  "dateOfServiceDefaulted": false,
  "coverage": { "coverageId": "U20252026012025010120261231", "active": false,
    "effectiveDate": "2025-01-01", "endDate": "2026-12-31" },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852556-60202" }
```

Covered on the first day, not on the last: the flag is false, the message says where coverage ends, and the period is still shown so intake sees how far it goes.

400 An unusable date of service is refused, never answered for today

```
{ "memberId": "T20262026", "dateOfService": "10/15/2027" }

{ "error": { "code": "INVALID_REQUEST", "message": "request rejected",
  "details": [
    { "field": "dateOfService", "code": "DATE_OF_SERVICE_INVALID",
      "message": "expected a real date in yyyy-MM-dd" } ] },
  "correlationId": "ONYX-PA-2026-000123",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13" }
```

Answering for today instead would read as "active" for a date nobody asked about.

8 Timeouts, retries, security

- **Timeouts.** A successful answer normally takes well under a second. When a lookup behind the service cannot answer, the failure is reported within about 15 seconds; set the client timeout above that.
- **Retry policy.** 503: wait the `Retry-After` seconds and retry; the number of attempts is to be agreed with Point32Health (three until then). 502 and 500: retry once. No response at all (connection refused, timeout): retry once. Never retry a 400, 401, 403 or 422. Every retry carries a new `requestId`. When the retries are used up: for 502, 503 and 500, park the transaction and alert the service owners with the `correlationId` and `requestId`; for no response at all, pass the member id to the UM vendor exactly as the EMR supplied it and record the `requestId` so the transaction can be reconciled later.

- **Security.** Authentication, certificates and rate limits are handled by the API gateway in front of the service; the credentials come with the base URL. The member id and date of birth are PHI: send them in the body only, never in a URL, and do not log the date of birth.
- **Caller identification.** Onyx sends `clientId` `ONYX` and `clientType` `EXT` on every call, and a new `requestId` per call; support finds a call by that `requestId` or by the `traceId` in the answer.